The Wayback Machine - https://web.archive.org/web/20240214214812/https://herotech.ro/cansat.html

CanSat 2021

What is a CanSat?

A CanSat is a simulation of a real satellite, integrated within the volume and shape of a soft drink can. The challenge for the students is to fit all the major subsystems found in a satellite, such as power, sensors and a communication system, into this minimal volume. The CanSat is then launched to an altitude of a few hundred metres by a rocket or dropped from a platform or captive balloon and its mission begins: to carry out a scientific experiment and achieve a safe landing. - ESA

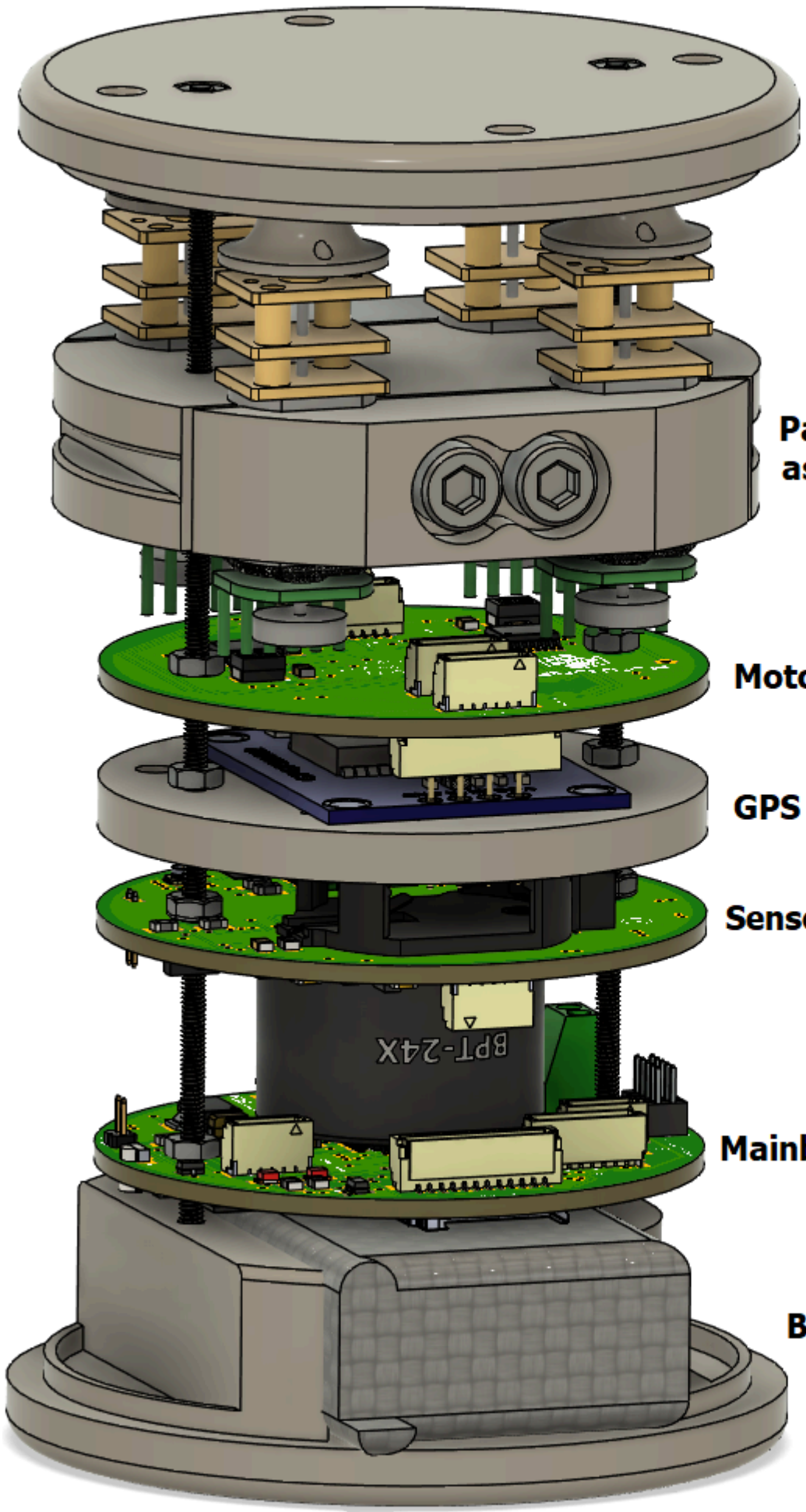




The mission we have chosen consists in a guided airdrop system. The CanSat should be able to guide itself during the fall by pulling the strings attached to the parachute with the target of landing in a nearby area relative to the dropping point. This presents a great potential in the development of an air drop package delivering system for faster and more convenient delivery which does not require human interaction. It is also a solution for dropping necessary items to people in need or for military drops. A CanSat of this size cannot carry much since it's light, but the project can be scaled to handle small items such as food packages, a medical kit and other similar-sized objects. The most important part of our mission is the control system. The exact position of our landing cannot be achieved with a lot of interfering factors, but we strive for stability. In order to call the mission a success we have the following goals: landing of the CanSat without damaging the carcass or the internal systems, correctly reading data provided by sensors for further analysing and guiding the CanSat for landing with high accuracy in the designated area.

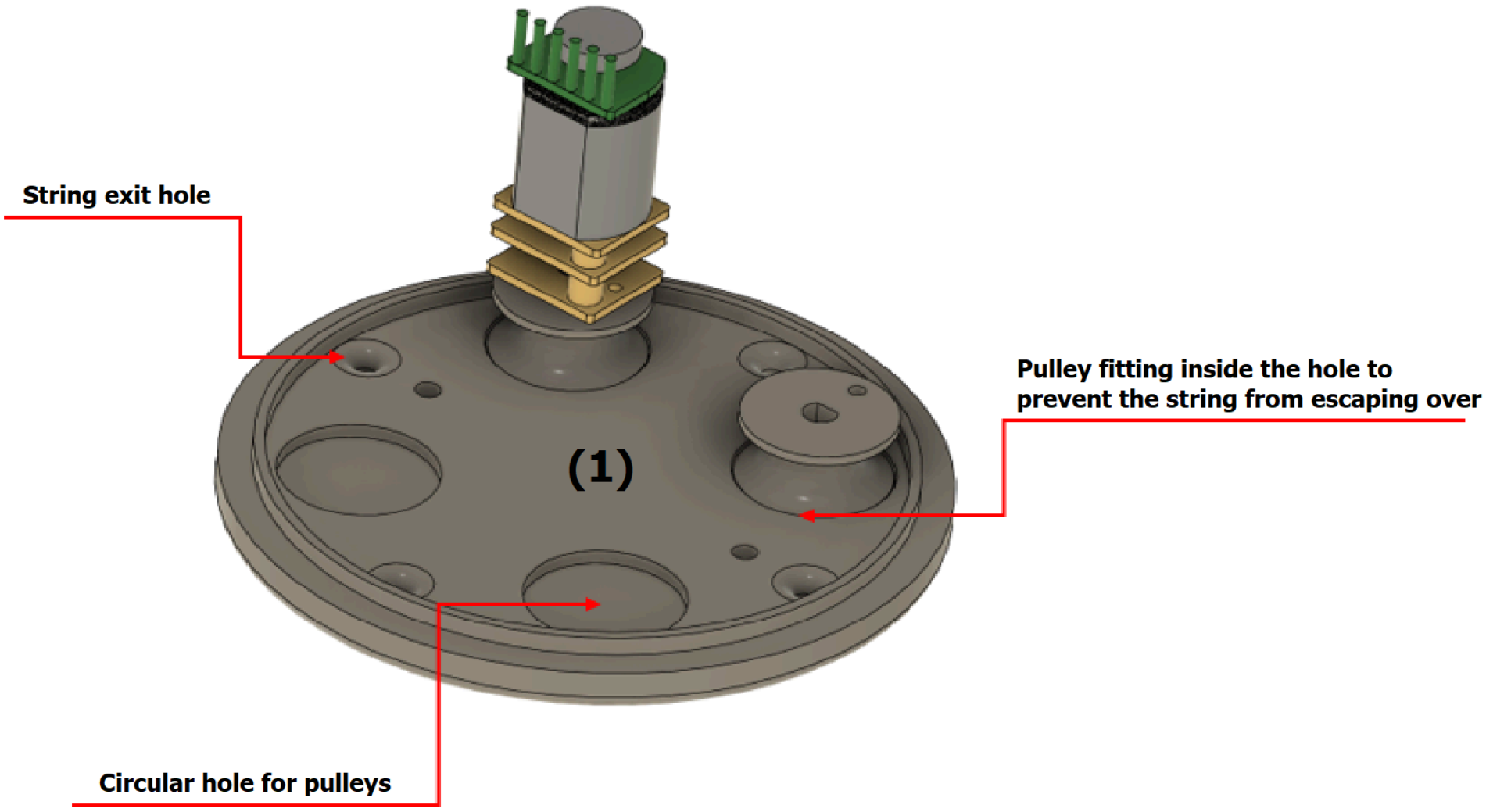
The CanSats mission objective is to collect data while descending with the help of a parachute and land in a designated area having a radius of no more than 25 meters. The mission begins with the CanSat being in a state of low battery usage. After being thrown from an altitude of 1km, its parachute immediately opens. Data from the accelerometer is continuously being sent to the main board, which will detect the beginning of the fall, switching all other functions on. While falling, the mainboard saves data on a secure digital card regarding altitude, temperature, pressure, geographic coordinates which are being read from a gps and a pressure sensor. For the CanSats navigation, the global position at a point in time is determined by the gps. The CanSats forward vector is determined by a magnetometer device whose tilt is compensated by the accelerometer and gyroscope. Comparing the current position to the target landing destination the CanSat will determine which direction it should face, and 4 motors will control the parachute to go to that direction by extending or retracting the ropes connected to the parachute. After landing the CanSat will switch to a low power mode, stopping data collection and turning on a buzzer to facilitate its recovery.

Mechanical and structural design



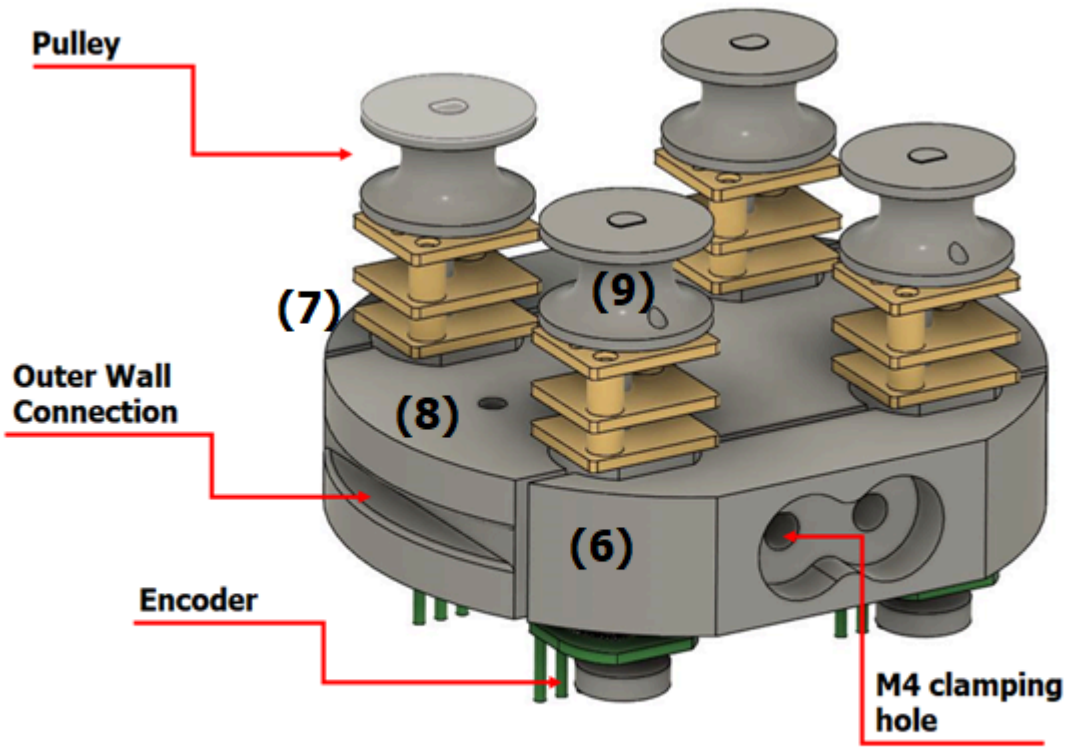
The CanSat was firstly designed using Autodesk Fusion 360, where all the 3D printed parts were modelled. We started by creating a maximum volume to make sure that everything we design was in parameters. The first step was to place all the boards on top of each other, leaving as little space as possible between them. The second step was to find and buy a suitable battery to fit in the CanSat. We decided to use one LiPo 800 mAh 7.4 V accumulator. Next, we started modelling the parts necessary to close and protect the circuitry, keeping in mind the fact that they had to hold up at 20G of acceleration. After all the required parts were modelled, we started printing the using PLA as we decided that it's properties would best suit the purpose of the CanSat and assure proper strength. We were aware that the GPS would run at quite a high temperature, so we measured it (70° C) and it was too high for a PLA board to not deform at that temperature, especially at a high G force during launch. Due to this fact, we decided to use ABS to print it which is a better thermoplastic than PLA when it comes to temperature resistance. When all the pieces were put together, we used the remaining space to incorporate the Parachute control assembly. The motors have a gear reduction of 210 to 1, giving us a high torque to pull the strings of the parachute. After printing all the parts, we assembled the CanSat using two M2 threaded rods that go through the whole CanSat and keep all pieces in place. For extra protection, the CanSat is covered in a two-piece wall that also connects to the Parachute control assembly to provide more stability. We are using plastic M2 nuts with 3D printed spacers between the boards to be sure we are not tightening the nuts too hard, damaging the PCB. This assembly forms the upper part of the CanSat and its purpose is to steer the parachute, allowing us to attempt a guided landing. We are using 4 DC micromotors, each one of them connected to the body of the CanSat with 3 3D printed parts that clamp them together with the use of two long M4 Screws. The printed parts are secured to the rest of the CanSat in 4 points: 2 by the M2 rod and 2 by pressing the sides in the outer walls, providing a tight fit and ease of removal. To each one of the motors, 3D printed pulleys are attached that allow the length of the strings of the parachute to be completely controlled. With this in mind, we can completely steer the parachute, which is a paraglider, by pulling on the side strings, or even slowing the descent rate by pulling all the strings simultaneously. The motors are provided with encoders that we use to make sure the parachute is correctly controlled.

1. The parachute control assembly



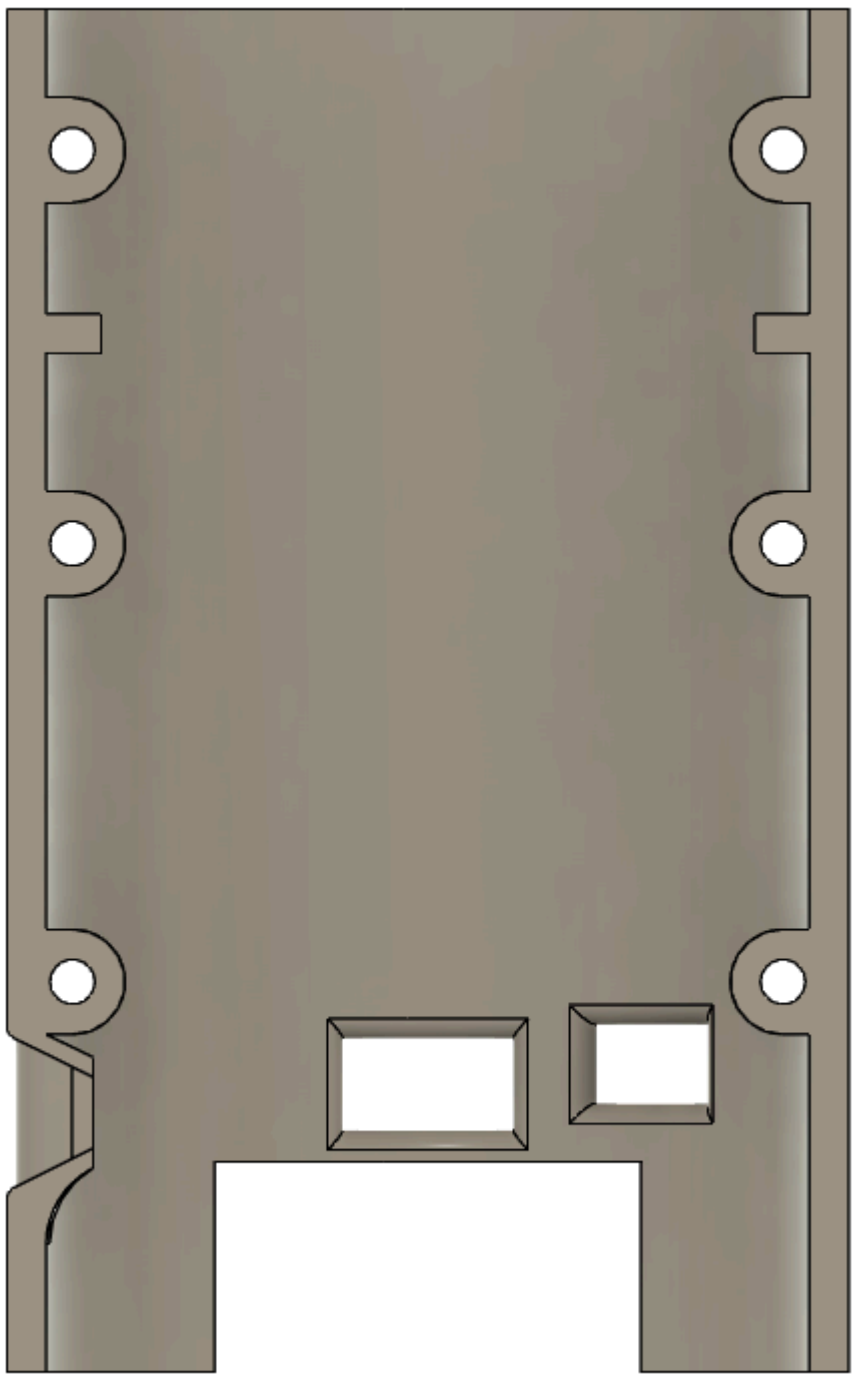
The printed parts are secured to the rest of the CanSat in 4 points :2 by the M2 rod and 2 by pressing the sides in the outer walls, providing a tight fit and ease of removal. To each one of the motors, 3D printed pulleys are attached that allow the length of the strings of the parachute to be completely controlled. With this in minde, we can completely steer the parachute, which is a paraglider, by pulling on the side strings, or even slowing the descent rate by pulling all the strings simultaneously. The motors are provided with encoders that we use to make sure the parachute is correctly controlled.

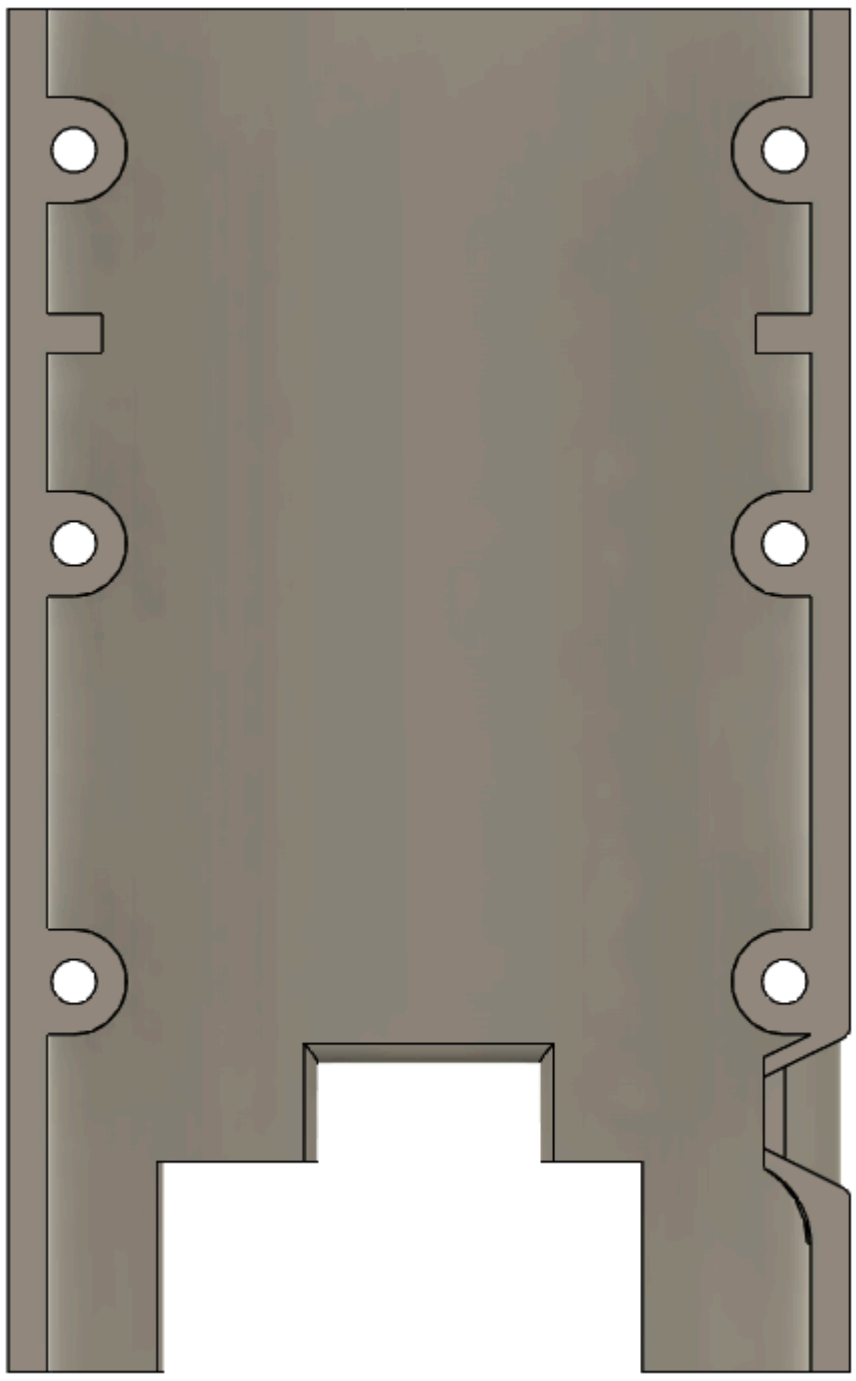
This assembly forms the upper part of the CanSat and its purpose is to steer the parachute, allowing us to attempt a guided landing. We are using 4 DC micromotors, each one of them connected to the body of the CanSat with three 3D printed parts that clamp them together with the use of two long M4 Screws.



2. The outer walls

These two walls have the sole purpose to stabilize the CanSat and keep the circuitry safe.

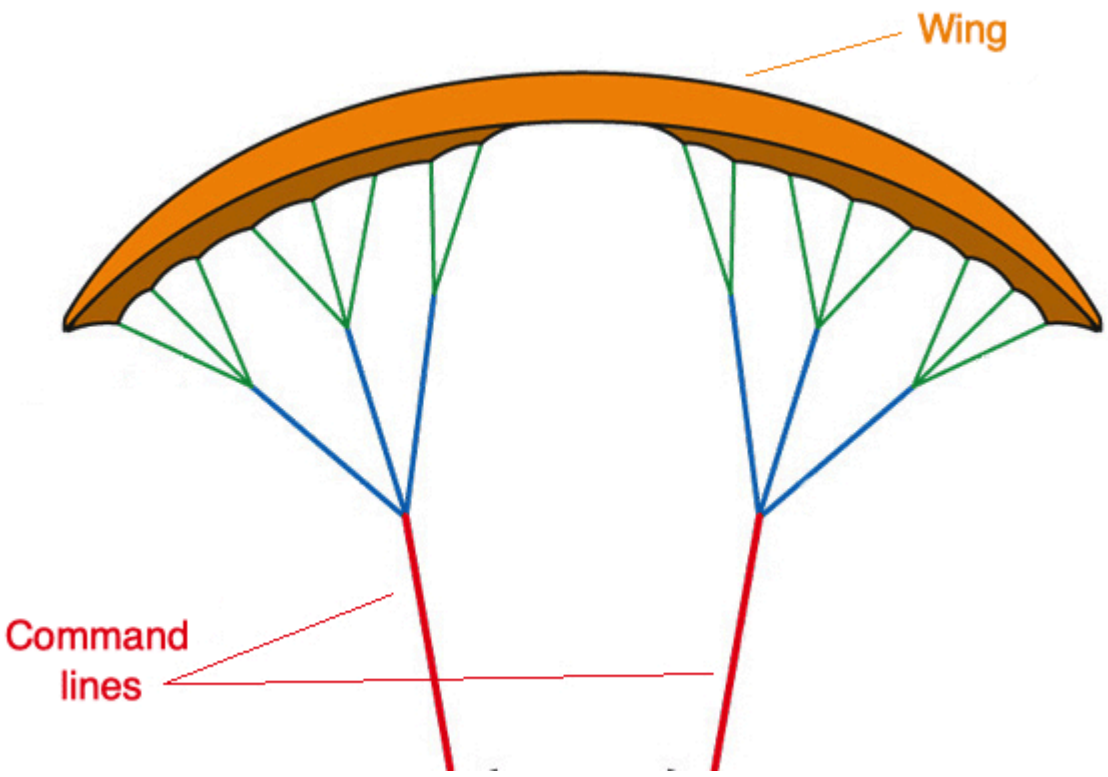






The recovery system

To recover our CanSat after deployment we decided to use a paraglider, which plays a big part of our secondary mission, where we are attempting a directed landing. Using the command lines of the paraglider which are connected to the Parachute control system, we are going to steer the CanSat. It will be attached to the CanSat through the 3D printed top in multiple points (in dedicated holes, around the motor axes) to make sure they are secure. The paraglider will be made from a light, impermeable material.



The steering is done as follows:

- By pulling on the left line, the parachute steers to the left
- By pulling on the right line, the parachute steers to the right
- By pulling on both lines at the same time we can decrease the drag coefficient, increasing the velocity of descend.

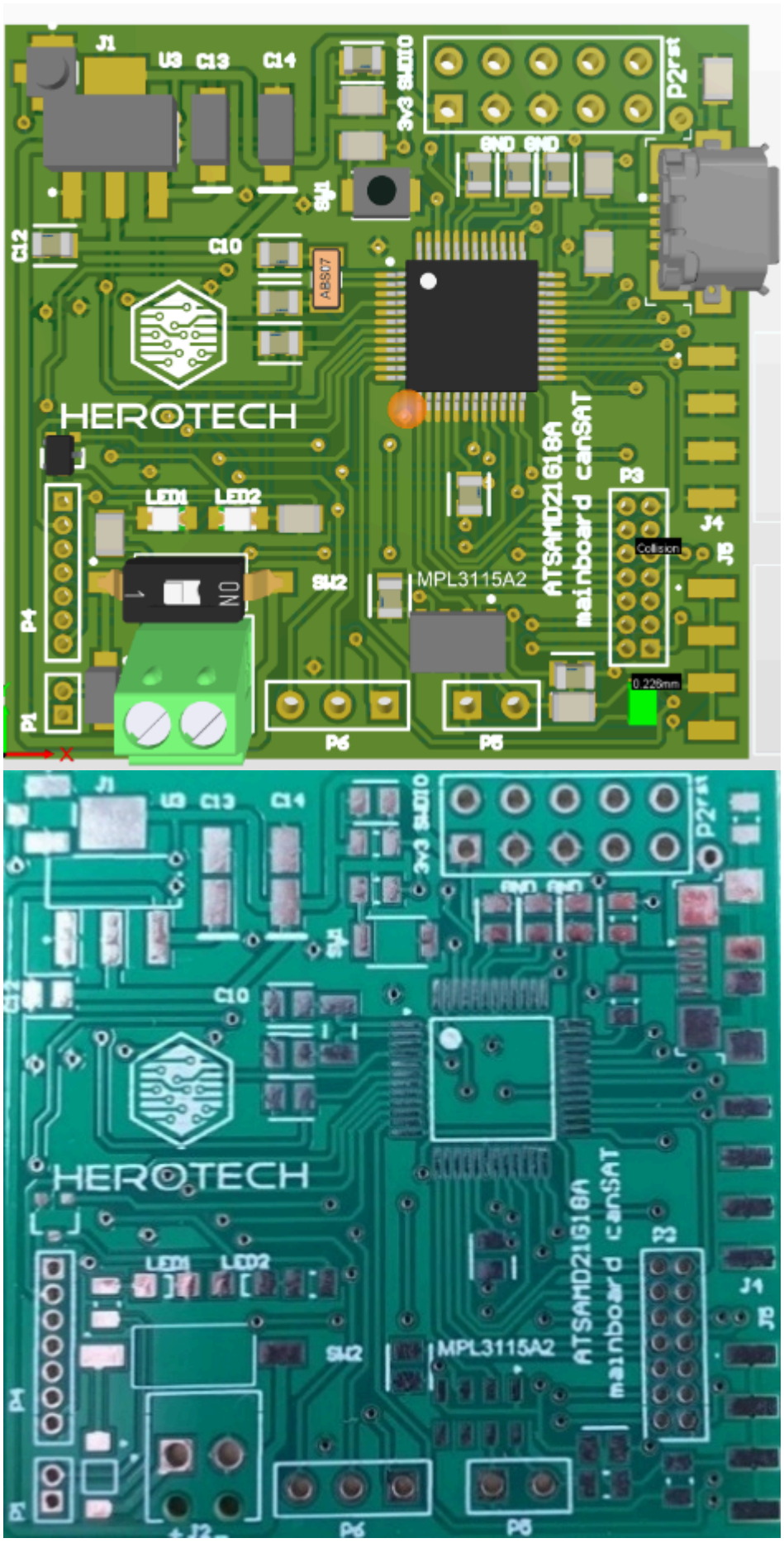
We calculated the necessary area of the parachute, getting the value of approximatively 0.085 square meters to reach the target velocity of 8 meters per second, with an approximate flight time of 150 seconds. With this in mind we started designing the parachute, which proved to be an incredibly difficult task due to the complex airfoil shape of the paraglider, and the necessity to be made of multiple cells. At the moment of writing, we decided that the first parachute we designed in Fusion 360 is a bad idea and we are waiting for a toy kite shaped like a paraglider to test.

The mainboard

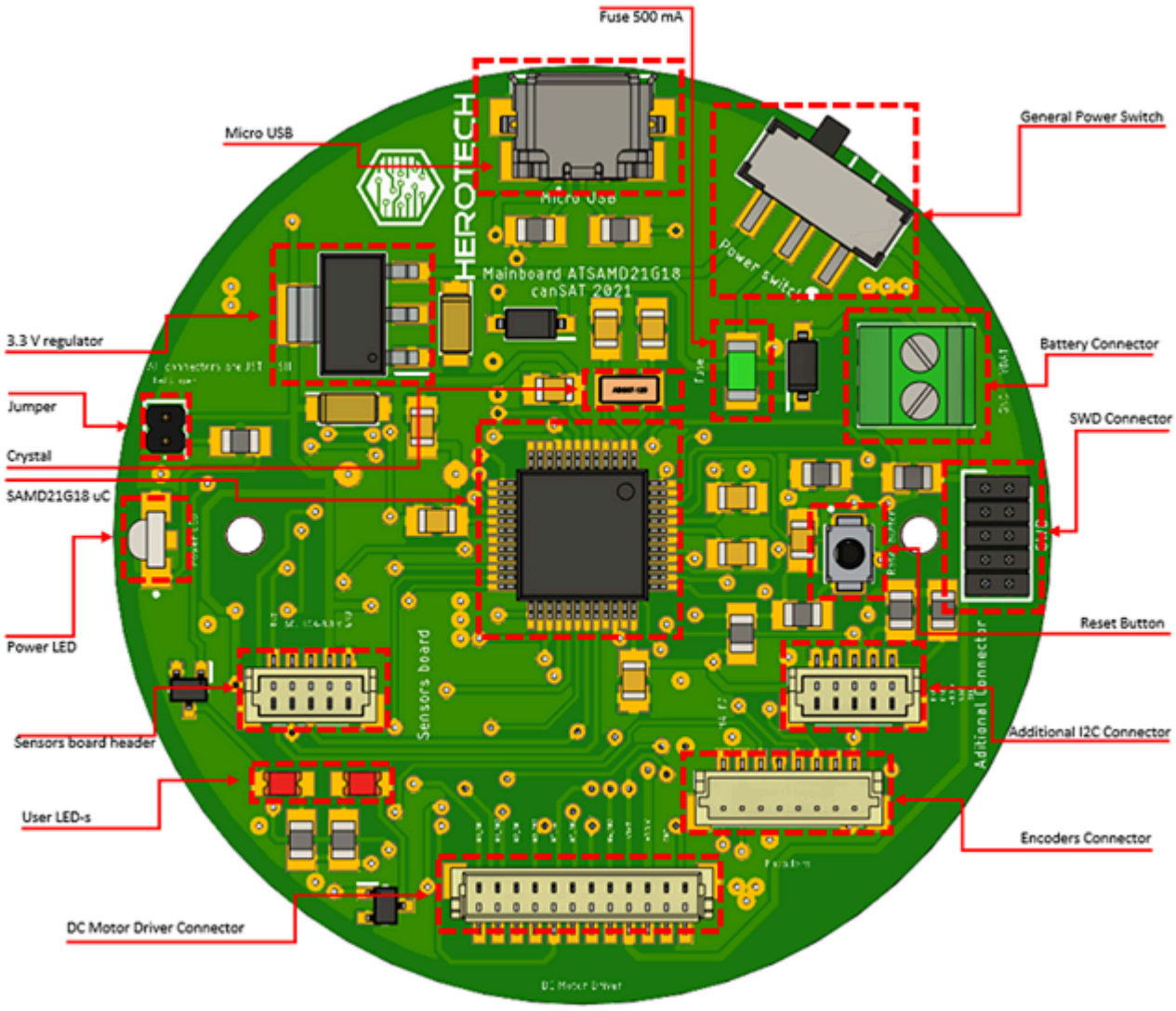
We started with the idea of developing our own printed circuit boards in order to make a customized application with the appropriate requirements. After a well documentation on the available microcontrollers, we decided to use an Arm Cortex M0 processor, being one of the smallest 32 bit processors available on the market. The decision was based on its performance, power consumption, programming accessibility and price. We use the ATSAMD21G18 microcontroller which is ideal for a wide range of automation and industrial applications. It has 256 KB of flash and 32 KB of SRAM with an operating frequency up to 48 Mhz. One of the main advantages of this family of microcontrollers is the serial communication module (SERCOM), configurable as UART, SPI and I2C. Another characteristic that was crucial for our application was the PWM generator capability, having 20 PWM configurable channels. The power supply range is between 1.62 V and 3.63 with a low power consumption. The idle and sleep modes are available for this chip as well as many others useful features for a complex embedded system. It comes in a variety of 48 pin packages, but we chose the TQFP one for soldering purposes.

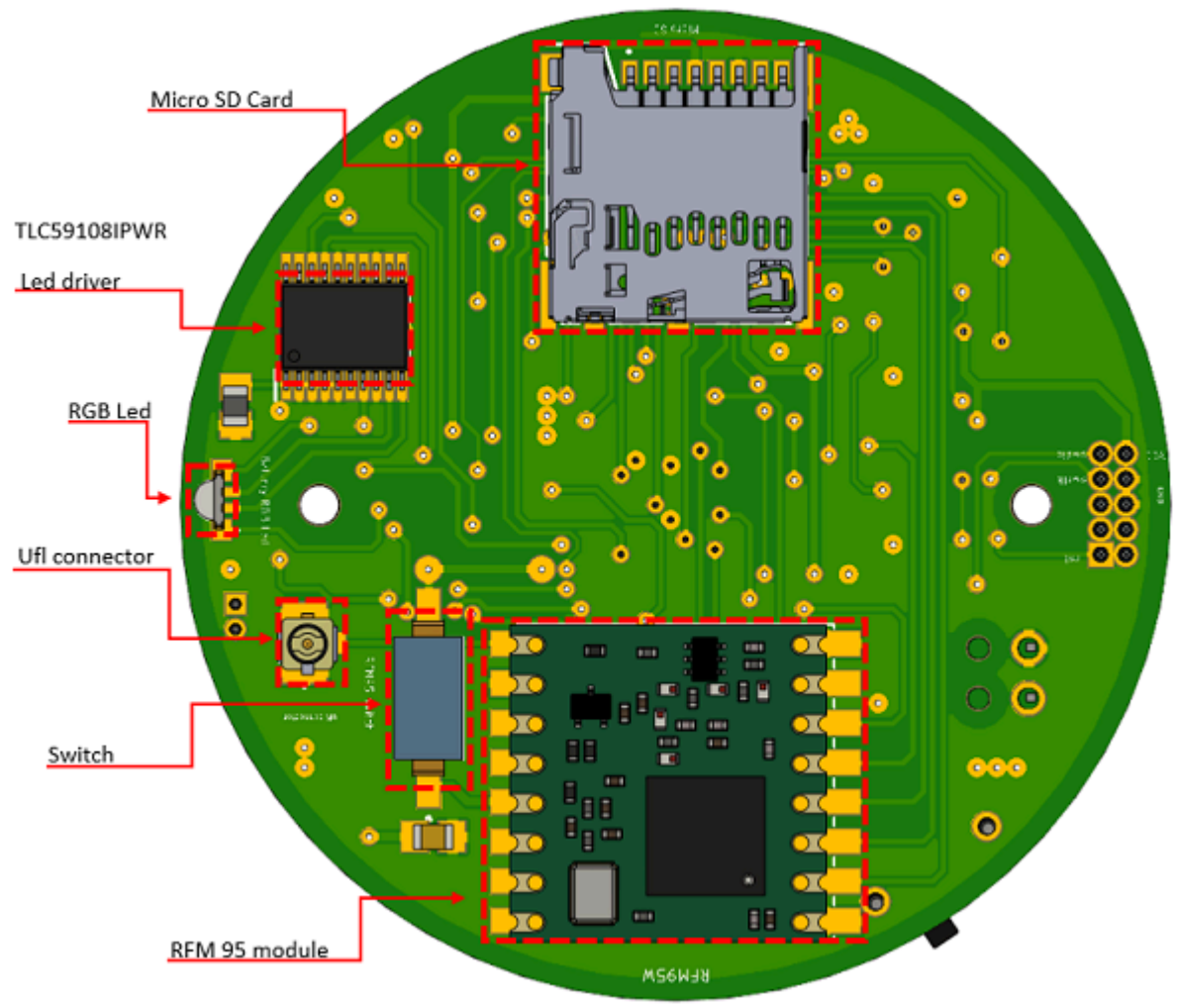
Initially we planned to use the microcontroller on a customed board, but when we realized that the programming part was quite difficult (the available libraries were not compatible with the hardware), we decided to use the same design as an Adafruit Feather M0. In this way, the programming was more accessible because any available library could be easily implemented in our project. We use the same pins as the Feather M0 with small modifications.

After choosing the uC, we started designing the schematic of our project. Being just a test board, just the Uc, a radio communication module, and a temperature and pressure sensor were integrated as well as the required auxiliaries (the power supply, the micro USB, SWD). We left all the pun headers available for further applications on the test board. We designed it in Altium because we had a free student license at that time. It was a challenge for our team to work with such a performant design software, but in the end all the experience was totally a success. The schematic was designed accordingly to the information specified by the supplier in the datasheet. We followed the recommended typical application schematic and layout. After we designed the schematic and the layout of the PCB-s, we manufactured them with a specialized company. In all iterations of the board, we used 0805 SMD packages for the ceramic capacitors and resistors.



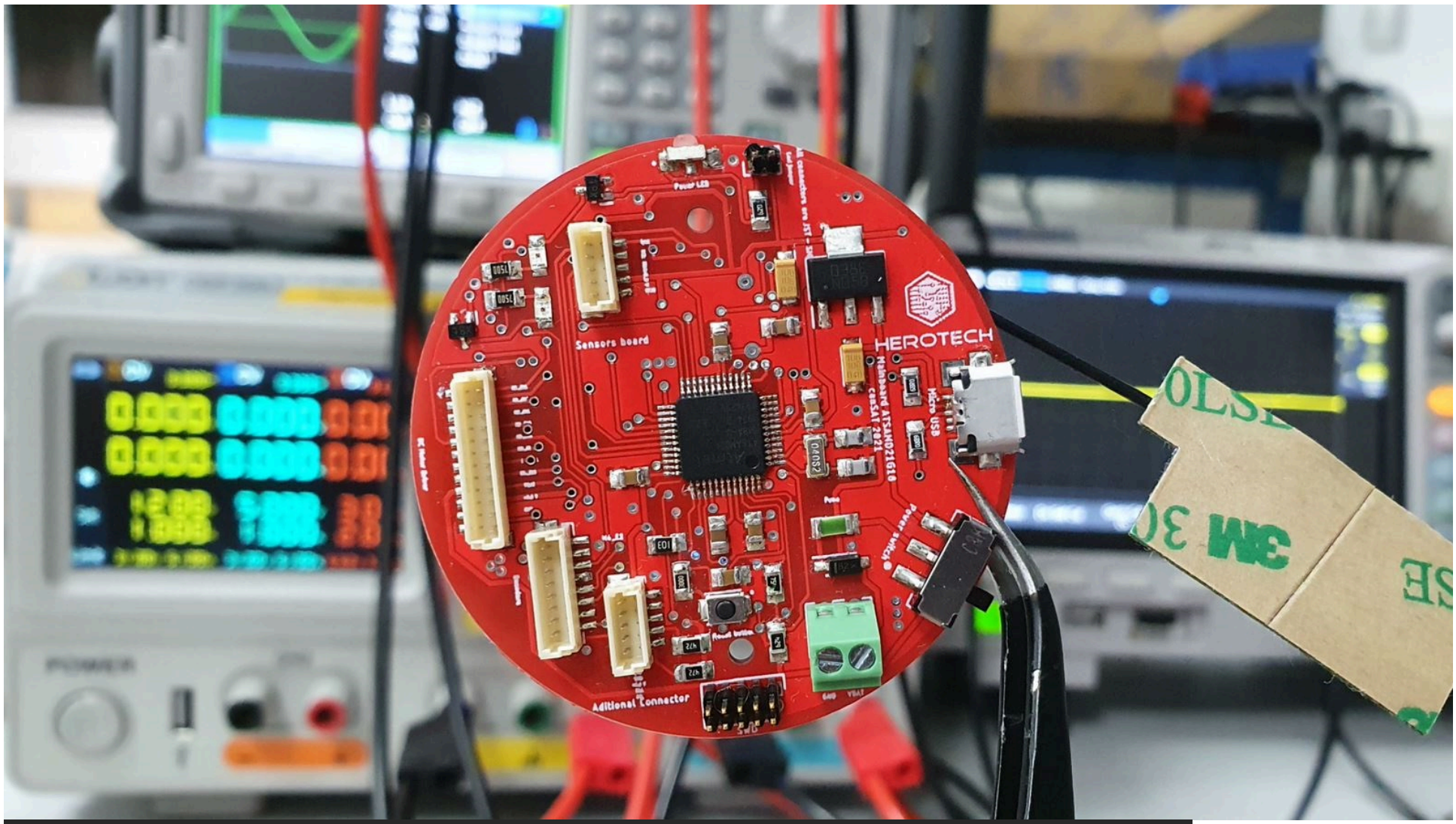
After populating the PCB, we burned the bootloader and did some tests on the board. The results were promising, so we started to redesign it considering all the CanSat requirements. The following descriptions of the mainboard are applicable for the second version.

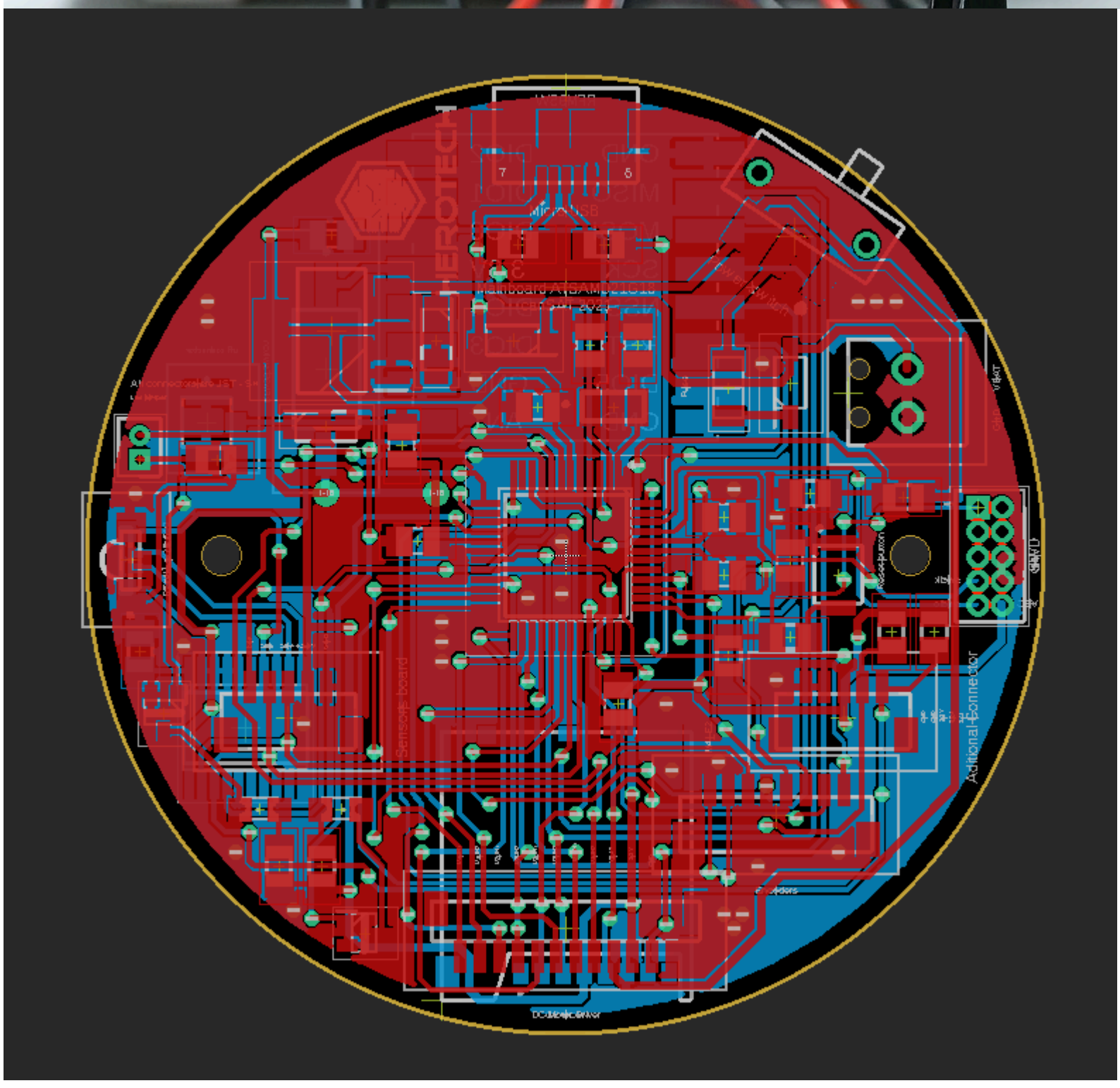




Layout of the mainboard

All the PCB-s used in this projects are double sided with the shape of a circle with 50 mm diameter. For mounting, we use 2 2mm diameter holes. All of the parts that have to be accessible by the user have been placed on the edge of the board. All the traces have been calculated for the appropriate current. The calculations have been made using specialized software. On both sides of the PCB, ground planes are integrated. The layout was designed in Autodesk Eagle, taking in consideration the recommendations available in the datasheets of the components. One crucial aspect in designing the mainboard was placing the bypass capacitors as close as possible to the uC.



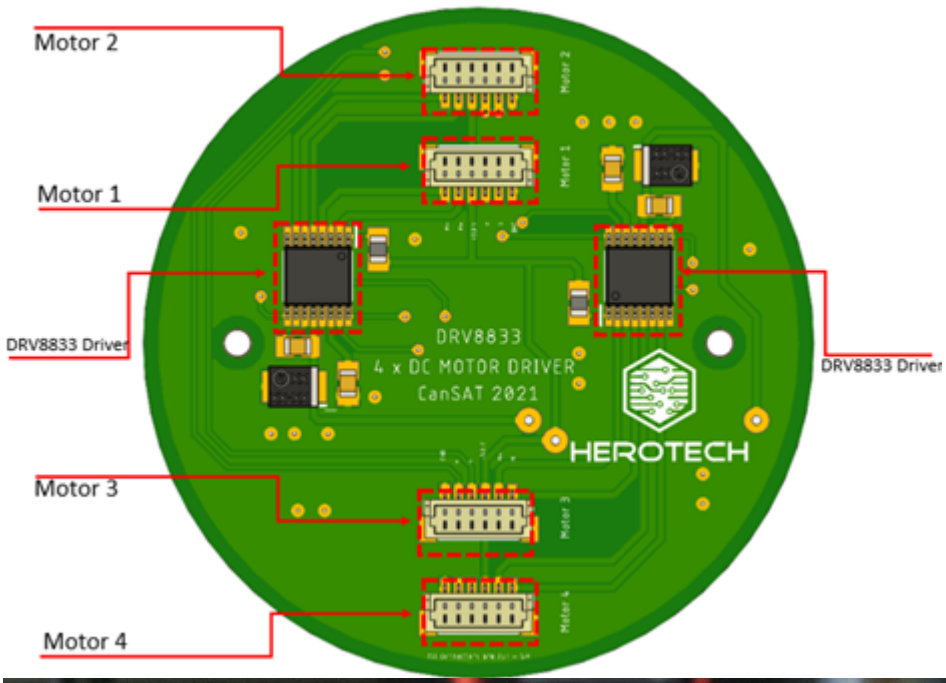


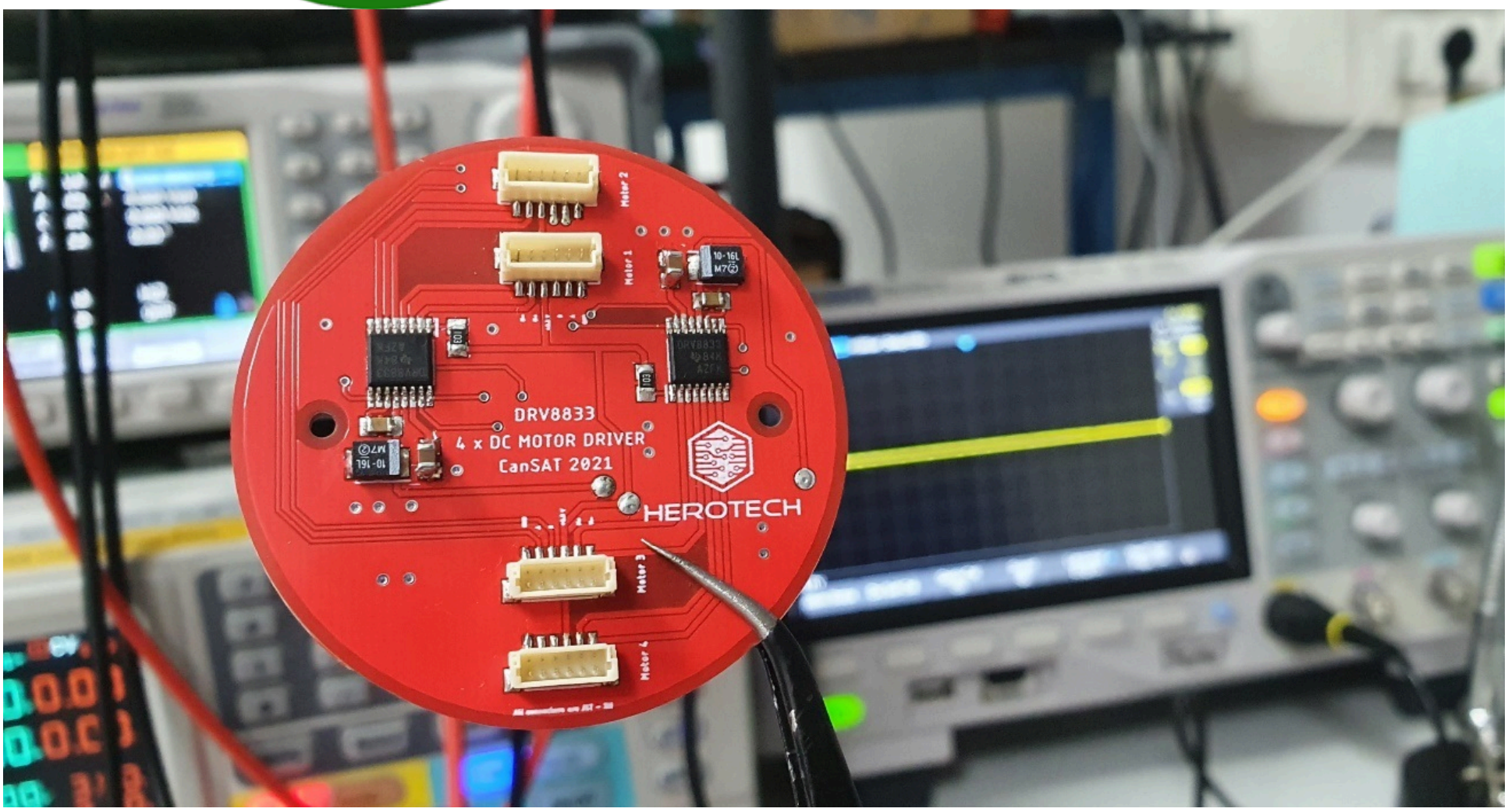
The DC motors Driver Board

We are using two DRV8833 Dual H-Bridge Motor Drivers in TSSOP package. This driver is very simple to use, and our team already has experience with it. It has a wide power supply voltage range (2.7 V-10.8 V), PWM winding current regulation and current limiting. Also, it comes in a thermally enhanced SMD package that does nor require external heatsink.

It provides a dual bridge motor driver solution, being able to drive two DC brush motors or a stepper motor. In our application, it is used to drive 2 DC micro motors from Pololu 210:1. These motors are reliable because they have a low current consumption (max 650 mA) and a 6V operating voltage. The motors are dual shaft, so on one of them we mounted quadratic encoders in order to control them precisely.

On the board, we are using 2 drivers in order to control four motors which are directly connected to the board via JST pin headers. The encoders are mounted on the motor shafts and the connection to the microcontroller is done using the same connector used for controlling the motors.



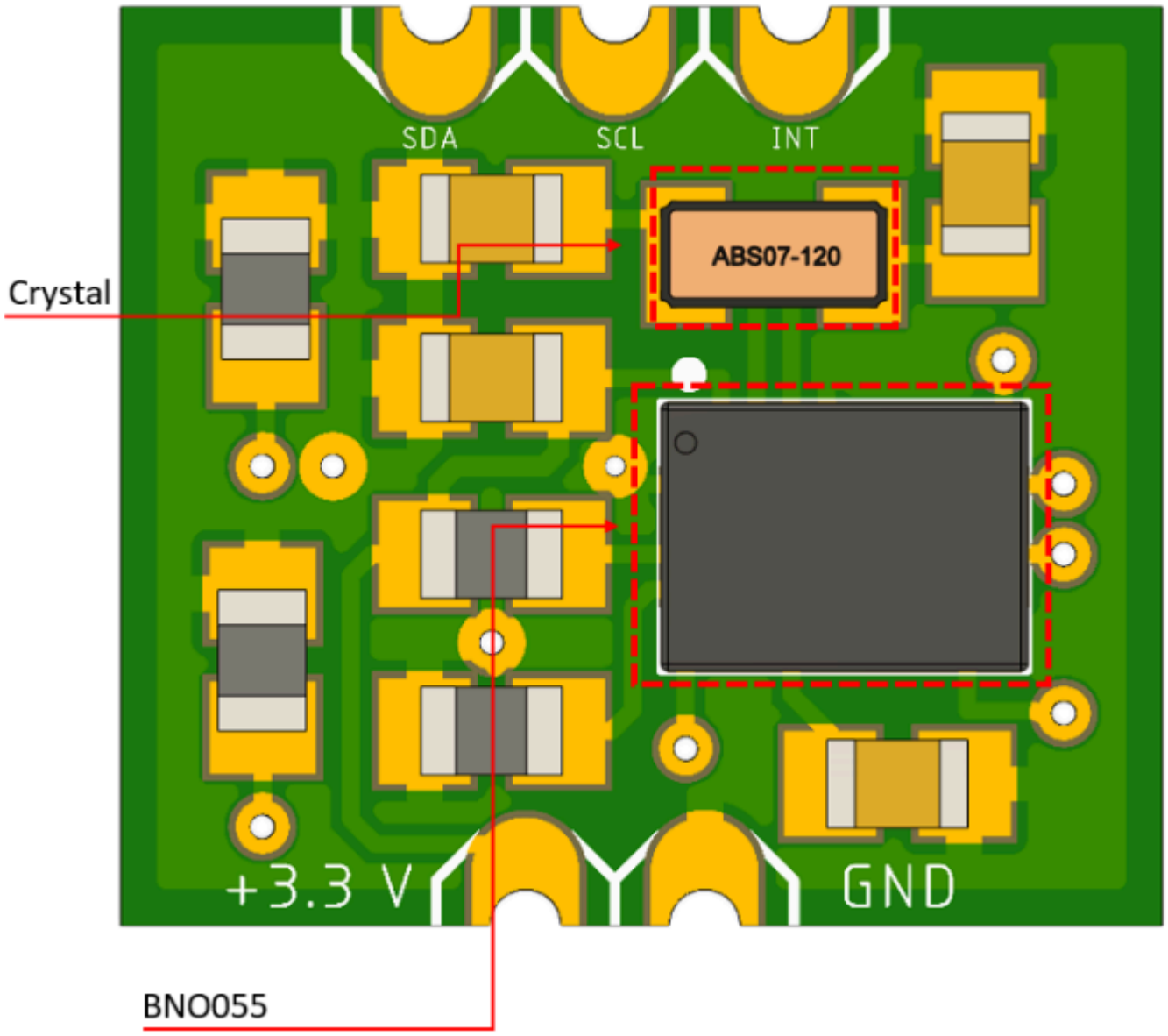


The sensors board

Initially we started from the idea that every sensor should be easily removed off the board in case of damage and replaced with other one.

First iteration

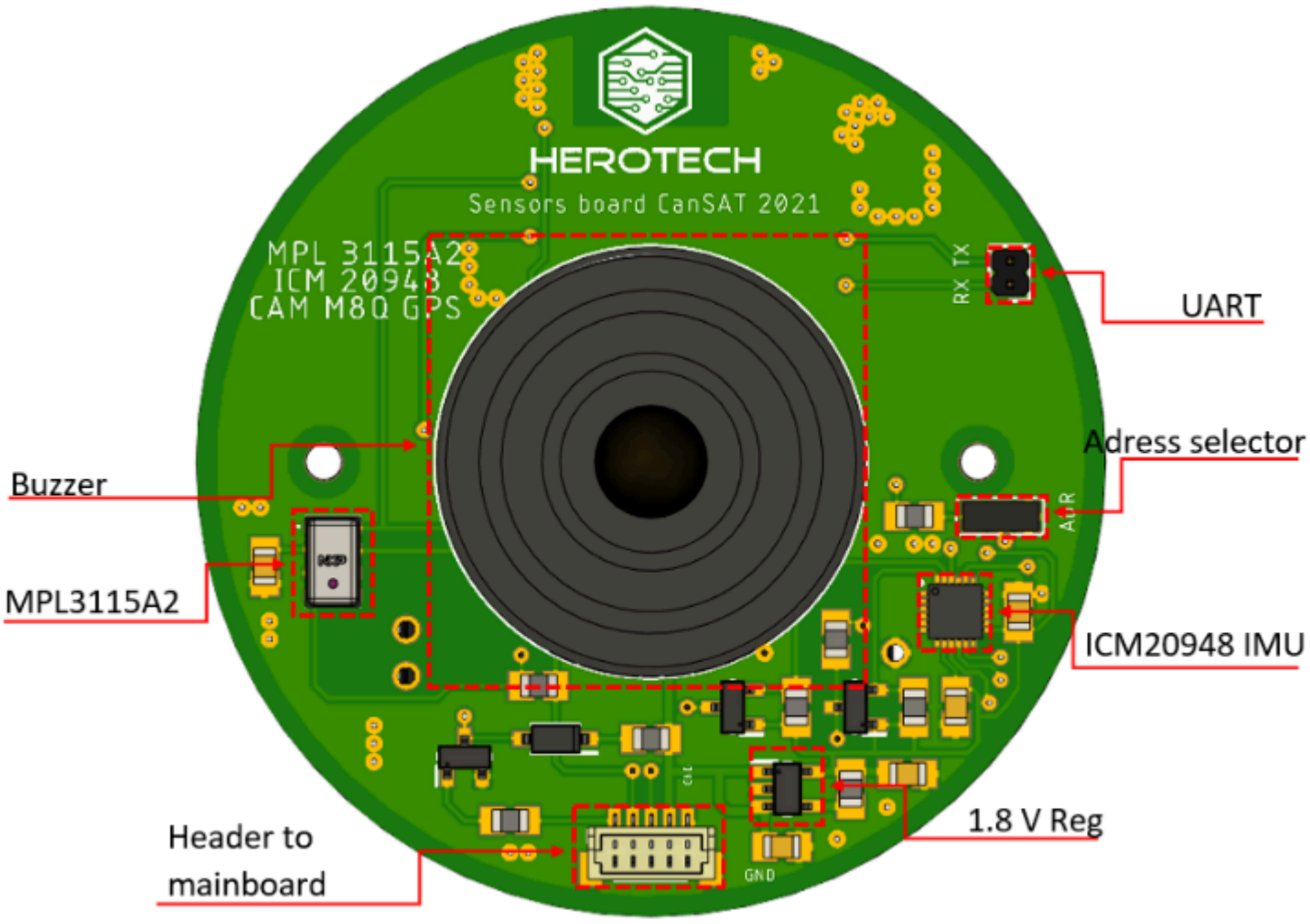
For the temperature and pressure sensor we use the MPL3115A1 from NXP, BNO055 from BOSCH as an IMU and an u-Blox SAM M8Q as a GPS. The IMU and the temperature and pressure are designed in a small PCB-s with castellated holes on the edges. These are mounted on the board that contains the GPS module. The schematic for these two small boards are really simple, they require just a few capacitors for filtering and some pull-up resistors. The temperature sensor was easily solderd on the board using a hot air station, but the BNO055 was quite difficlt to solder because it comes in a small package and after the soldering, we noticed few short circuits between the pads under the chip. Although we managed to solder it properly, we did not have the proper equipement for the soldering process, so we decided to use another IMU in the next iteration of the sensors board. Unfortunatelly, we did not manage to solder the GPS module properly because it has pins on a big area and the internal components were damaged during the porcess.

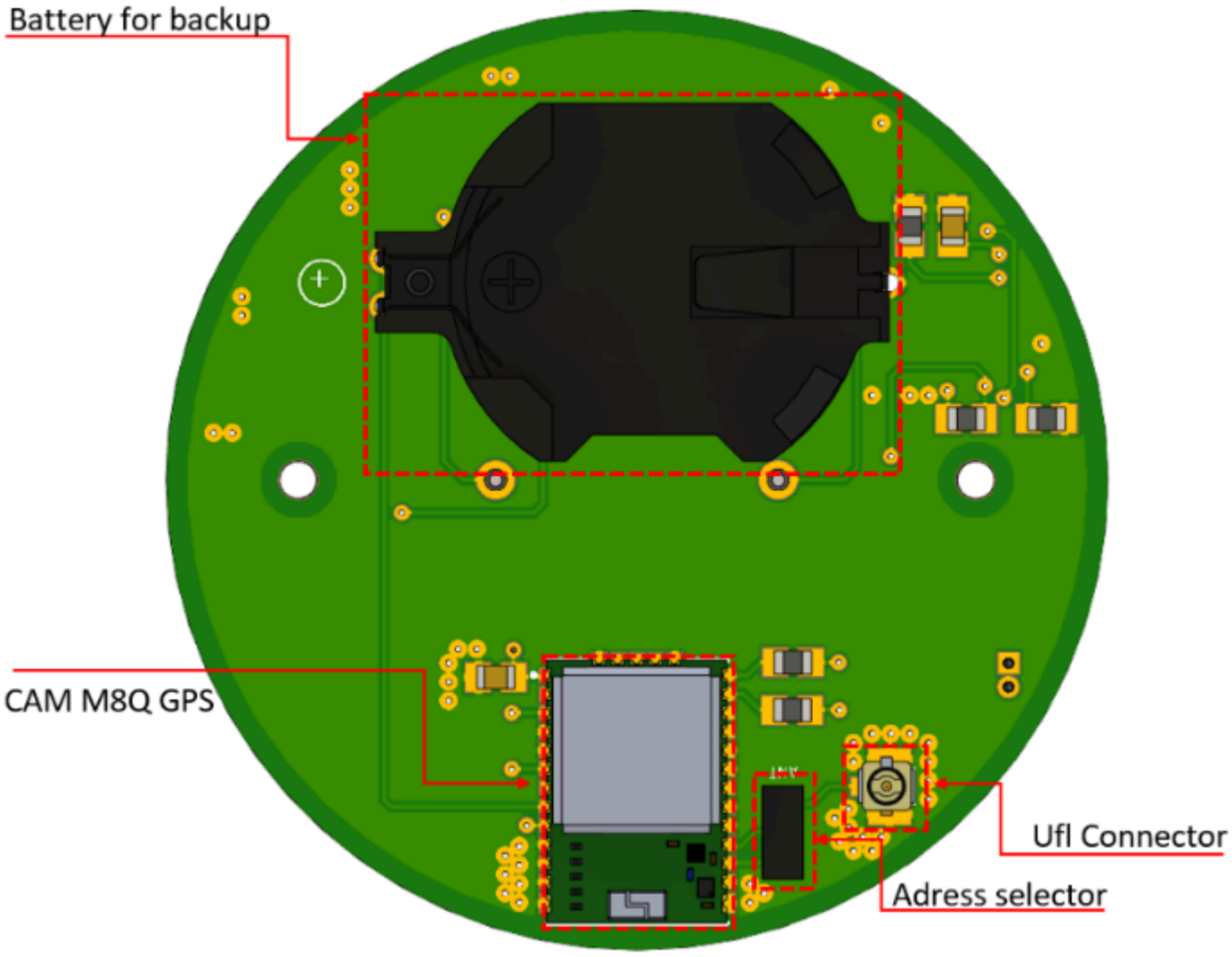


Second iteration

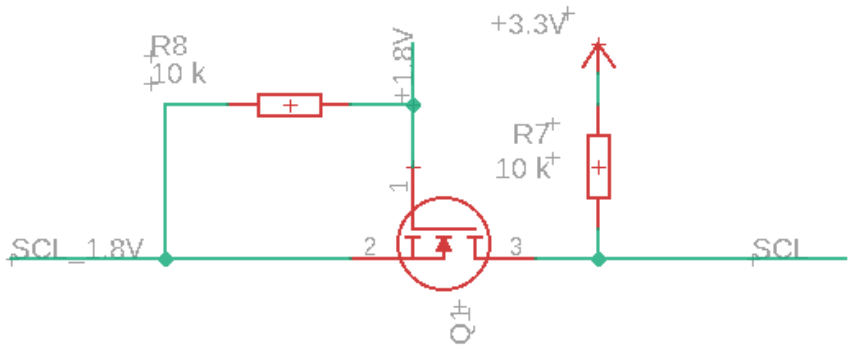
We developed a new sensors board with another GPS module. The temperature & pressure and the IMU remained the same as in the previous concept. As for the GPS module, we used the CAM M8Q from u-Blox. Because we wanted to accelerate the process of making our new version, we prototyped the PCB in house using photosensitive PCB and UV light for curing. After the UV exposure, the PCB was introduced in a special pcb developer sodium hydroxide solution an then in Ferric Chloride (FeCl3) for etching. Unfortunately, we did not take in considerations the antenna requirements, so the module was not able to view any satellites.

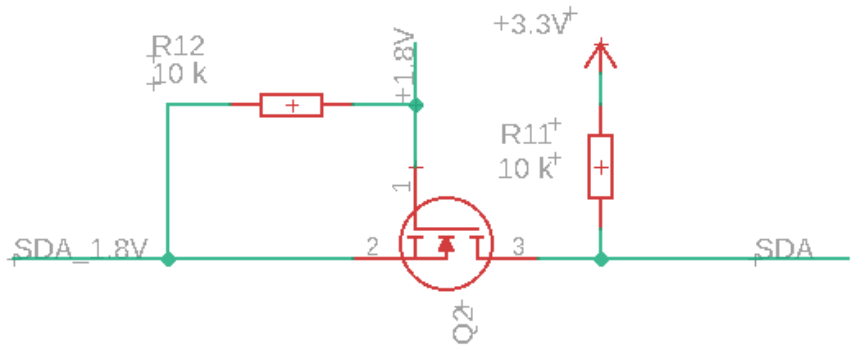
Third iteration





This version came with a proper layout for the GPS module and another IMU. We chose to replace the BNO055 IMU with ICM 20948 for soldering purposes. The only issue was the operating voltage of this IMU: it works with 1.8 V, fact that forced us to use a 1.8V regulator and a logic level converter for the I2C interface. MCP1824 1.8 Vreg is used as a regulator and it provides high current (300 mA), short circuit current limiting and overtemperature protection. The logic level shifting is achieved by 2 N-Channel MOSFET Transistors (BSS138LT7G) placed on SCL and SDA lines. They come in a small package and the threshold voltage is ideal for our application (0.85 V – 1.5V). Also, in this version we integrated a battery for GPS backup. In each iteration the buzzer circuit remains unchanged. Basically, it is just a transistor that acts like a switch, opening the circuit when on base is applied a logic high.





Fourth iteration

A fourth iteration of the sensors board was necessary since we were not happy with the result of the CAM-M8Q's output of data. Inside, the GPS would not be able to receive data from satellites, and when tested outside it would only work sometimes. We decided to test another GPS that we had in the laboratory: the u-Blox Neo-6M. This GPS offered much better results since it had a much bigger and specialized antenna. We ended using this GPS since it satisfied our needs in term of accuracy and signal strength, but we faced difficulties while connecting it to our mainboard since we didn't have enough free pins. To have the GPS communicate through UART, the board needed to have 2 free pins on the same SERCOM. This was made possible by giving up a led and a motor encoder.

Soldering and testing

All the soldering and testing process have been done in an ESD safe environment, in order to prevent damage of sensible components. After we soldered every PCB, we connected them together using proper connectors for the JST SH 1mm pin headers. Although it required special attention, we crimped our own wires using an Engineer PAD 11 crimper tool. After we made sure each sensor works and sends proper data, we tested the system by connecting it to a lab power supply with current analyzer and having the board request data from each sensor at a small interval of time. The results were close to those calculated on paper before, about 375 mA. After studying its behavior and confirming that the system is stable, we tested powering the CanSat with a LiPo battery with a capacity of 800mA and an output of 7.4V. With the battery fully charged and every sensor and motor in use the system stayed online for about 2.1 hours. On power saver mode, when the system only handled data from the IMU, a battery lifetime of about 10 hours was registered

Software Design

Our CanSats main board controller uses an ATSAMD21G18 chip. We burned a bootloader from Adafruit to it so that it could act as an Adafruit feather m0 board. This was possible because feather m0 uses the same ATSAMD21G18 as a processor. This helped us by opening the possibility to use an Arduino as an IDE to help write, manage and upload C++ code to the board. Another advantage is the already defined configuration for the pins of the board, meaning that we could still use software interfaces such as Serial without having to change the registers or pins configuration.

The CanSat will have 3 phases corresponding to the state of the mission: before launching, launch phase and after landing. To automatically switch between the phases, we are using a state machine system.

Phase 1 – Before Launch. The CanSat will boot up the program at starting. It will calibrate the sensors and make the RGB led green if everything is working properly and the CanSat is ready to start its mission. A blue light is used to indicate a partial initialization, for example if a sensor fails due to different reasons. This is not ideal but the CanSat will be able to complete its mission partly. It is recommended to reboot from the switch if a blue light is showing. A red led is used when a fatal error occurs. This means that the CanSat will not be able to control its descent, or it will not be able to store data collected during it. This phase is working in a low power mode, reading data only from the accelerometer to determine when to switch to the next phase. We are averaging from multiple readings in order to validate the data and not switch to the next phase if the sensor outputs an erroneous value.

Phase 2 – Launch phase. This phase starts when the CanSat detects getting dropped out of the plane/helicopter. At this point the main board will read data from all sensors and will save them to the SD card at a rate of 200 MHz The read data consists of relative time since start, acceleration in the Z axis, pressure, altitude, temperature, longitude, latitude, battery voltage. At the same time, the yaw angle of our CanSat is calculated from the readings of the ICM20948 sensor. The CanSat will always move in the direction of the parachute's forward vector, so by knowing the yaw angle we also know in which direction the CanSat is heading. We calculate the direction in which the CanSat should be heading by subtracting the landing target coordinates from the current position coordinates and evaluating the arctangent of their division. For turning the CanSat to the desired heading we send commands to the motors in order to retract the ropes. The side where we retract the ropes will act as a pivot for the spinning motion, and the heading will be adjusted, moving the CanSat toward the targeted landing spot.

Phase 3 – After Landing. The CanSat will detect the landing with the help of the accelerometer, using the same concept as the for the launch. After that it will record the GPS position in order to compare it to the targets landing spot and will start the buzzer. It will disable all sensor readings, since the mission has ended and data collection is no longer needed, and since power should be conversed for the battery to last until the CanSat is recovered.

MPL31152A2

This sensor is used to read the air pressure with the help of Adafruit MPL31152A2 library by getting the averages of more sensor's readings. The output in Pa is used to calculate the altitude in meters with the following formula:

$$h = (1 - \frac{p}{p_0})^{0.1902632} * 44330.77$$

$$h = altitude, p = current\ pressure, p_0 = pressure\ at\ ground\ level$$

TLC59108

We did not find a library for this RGB led that is compatible with Arduino, so we wrote our own by consulting the datasheet. In our program we define an object of type TLC59108 to which we pass the reset pin in the constructor. We defined different methods for this object such as setColor(r, g, b) to which we pass 3 values from 0 to 255, each representing the intensity of the red, green and blue channels.

ICM20948

To read data from this IMU we are using the Sparkfun 9DoF IMU Breakout Arduino Library. By enabling the Digital Motion Processor, the library is able to output the orientation calculated from the fusion of the sensor data from the 3-axis gyroscope, 3-axis accelerometer and 3-axis compass in either Euler or quaternion format. For calculating the compass heading we could technically only use the magnetometer, but the output value would change drastically with a change in roll and pitch of the CanSat. We have to make up for that by plugging those angles in our equations:

$$x_{horizontal} = x * \cos(p) + y * \sin(r) * \sin(p) - z * \cos(r) * \sin(p)$$
$$y_{horizontal} = y * \cos(r) + z * \sin(r)$$
$$\theta = \operatorname{atan}\left(\frac{y_{horizontal}}{x_{horizontal}}\right)$$

x, y, z= magnetometer readings in μT

p, r = pitch and roll angles

θ = compass heading

u-Blox Neo-6M

This GPS module provides the global coordinates, UTC time and temperature with the help of the TinyGPS++ library. On a cold start the GPS might take a few minutes to acquire the data broadcasted by the satellites in order to calculate a fix.

Dc Motors

Dc motor control is done via 2 digitals pins per each motor to be able to rotate the motor clockwise or anticlockwise. Each of the 4 quadrature encoders help us determine the absolute rotation of a motor since program start-up by being able to identify rotation counts in both directions. So instead of relying on a feed forward solution in order to actuate the motors, we have a closed loop solution which should be able to generate better results for our descent control system.

Budget

#	Product	Quantity	Price (Euro)
1	ATSAMD21G18	1	4
2	ABS07-120-32.768KHZ-T Cystal	1	1.5
3	JST Headers and Connectors	NA	30
4	Segger J-Link Programmer	1	25
5	Buttons & Switches	3	6
6	Micro USB Connector	1	2
7	Micro SD Connector	1	3
8	RFM 95W	2	50
9	uFL connectors	2	2
10	TLC59108IPWR	1	2
11	User LED-s	2	1
12	RGB LED	1	1
13	LM117 3.3 V regulator	1	1
14	Capacitors and Resistors	NA	5

#	Product	Quantity	Price (Euro)
15	Diodes	3	1
16	Transistors	5	2
17	Neo 6 M GPS	1	15
18	ICM 20948	1	8
19	MPL3115A2	1	6
20	Buzzer	1	4
21	1.8 V regulator	1	1
22	DRV8833 driver	2	5
23	Sensors Board Prototyping	1	1
24	Mainboard Prototyping	1	1
25	DC Motor Driver Board Prototyping	1	1
26	Shipping and Taxes - PCB Prototyping	1	20
27	Shipping - Electronics Components	3	50
28	Battery	1	10
29	LiPo Charger	1	20
30	Filament	NA	10
31	Wires and terminals	NA	5
32	Parachute	1	15
33	Mechanical Parts	NA	10
34	micro–SD Card	1	5
35	soldering (flux, paste, solder wire)	NA	3

About Herotech

We have the goal of developing as many technology related solutions as possible for improving everyday life.

Important Links

- Click here to email us.
- Help with a donation

Social Media

Copyright ©Herotech 2019-2021 - Template by Inovatik